

3.2 ppt

3.2 Planning 与任务拆解

P01 · 背景

把任务拆成可验收的子目标

- 本节主线：判断要不要计划 → 拆成可验收子目标 → 建立依赖与证据 → 用完成契约判断能否交付
- 案例：Coding Agent 修一次会话过期的边界问题

P02 · 任务一换，性质就变了

- 提示词只有一句话：

代码块

- 1 请修复登录模块的会话过期边界问题：先定位登录和会话相关代码，复现失败测试，
- 2 分析根因，只修改必要源码，然后运行目标测试、边界测试和回归测试，
- 3 最后把根因、改动和验证结果写入 `artifacts/login-fix.md`。

- 提示词只说是「会话过期边界」，没说错在哪一行，可能是：

- 日期粒度截断了时刻
- 时区语义
- 边界该不该包含到期时刻

这三条里哪一条成立，只能靠工具结果去证实或推翻。

P03 · Loop 当前问题：有对话，没有任务结构

- Loop 里保存的是一条**消息历史**：它知道模型读过什么、说过什么

- 它不知道**任务还差哪几步**
- 所以当工具结果推翻当前判断时，没有东西能回答：
 - 哪一步的结论作废了？
 - 哪些跑过的验证要重做？
 - 手上这份改动到底能不能交？

这不是模型不够聪明，是系统缺一份可检查的任务状态。

一句话：短任务，3.1 的 Loop 就够了；跨模块修复、故障排查、多轮研究这类长程任务，「做到哪了、凭什么说完成」必须由系统回答。

P04 · Planning 做什么

Planning 把任务拆成有完成标准、有依赖关系的子目标，再根据真实证据判断下一步和是否可以交付

- 新增：计划状态、计划操作、证据、完成契约
- 这四样东西，模型都做不了主，得由工程师定：
 - a. 计划存在哪里
 - b. 谁能改
 - c. 哪些步骤允许开始
 - d. 什么证据能证明完成

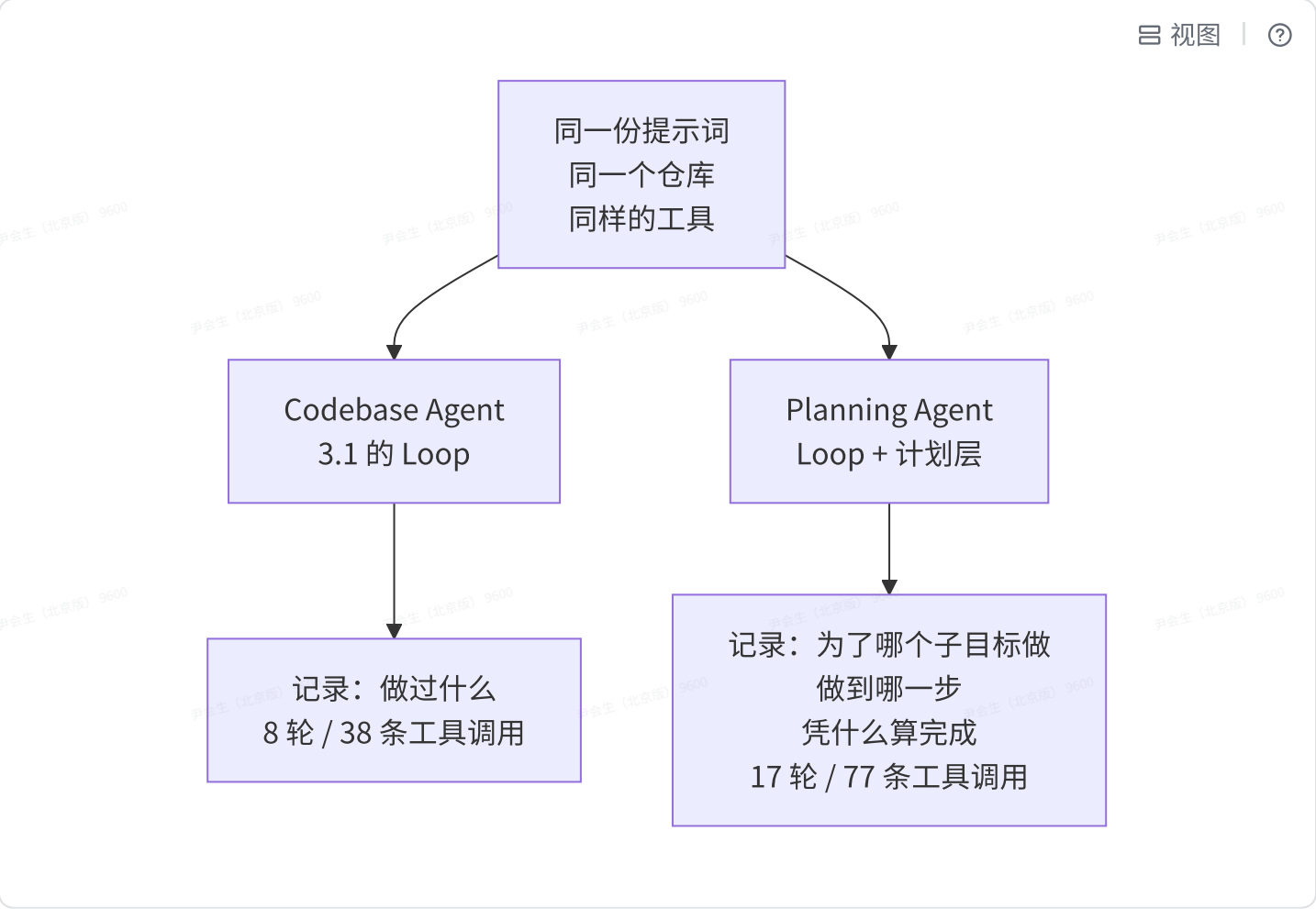
本节和「写一段更好的提示词」的区别——你要加的是一层工程结构，不是一句更严厉的嘱咐。

P05 · 怎么证明有用

- 给 Agent 加上计划层的 Planning

- 前提：对照组不是 3.1 原样——它的 Runtime 同样有 `apply_patch` 和 `run_test`，完成契约也已换成「当前代码版本上有改动 + 三个测试范围全通过 + 有交付说明」

示意图：



两者唯一的差别就是计划层，多出来的部分才能归因到 Planning。

P06 · § 一 要不要计划？ 1.1 两份真实运行的汇总

项目	codebase.md	planning.md
执行轮数	8	17
停止原因	COMPLETED	COMPLETED
工具调用记录	38 条	77 条
代码版本	—	r1
计划版本	—	v1

计划修订	—	0 次
步骤状态	—	s1-s6 全部 completed，证据 4/1/1/1/3/1

先别把「8 轮 vs 17 轮」读成效率对比。

Planning Agent 多出来的调用花在创建和读取计划、绑定步骤、补齐步骤级证据、分层验收上。

P07 · § 1.1 一句话差别

- Codebase Agent 记录了「做过什么」
- Planning Agent 记录了「为了哪个子目标做、做到哪一步、凭什么算完成」
- 38 次调用能告诉你调用了哪些工具，但回答不了：
 - 哪些调用服务于哪个子目标
 - 根因那一步完成了没有
 - 目标用例通过时，边界和回归跑了没有
- 日志里最典型的一段：

代码块

1 Evidence ev-4 belongs to s3 (not s1). Let me reference only s3-valid evidence.

- 模型随后重新读了一次源码，专门生成 `s3` 能引用的证据

计划层的价值不是「更少的轮数」，而是结论可以被解释、追踪和复核。

P08 · § 1.2 三层机制各管什么

层次	负责的问题	本节案例里的例子

Tool Calling	这一次调用哪个工具、传什么参数	读取 <code>session-policy.ts</code>
Agent Loop	工具结果怎么回写，下一轮怎么继续	读到失败日志后继续查实现
Planning	先完成什么、还缺什么、什么时候改变路径	先复现，再确认根因，再修复，再验证

示意图：

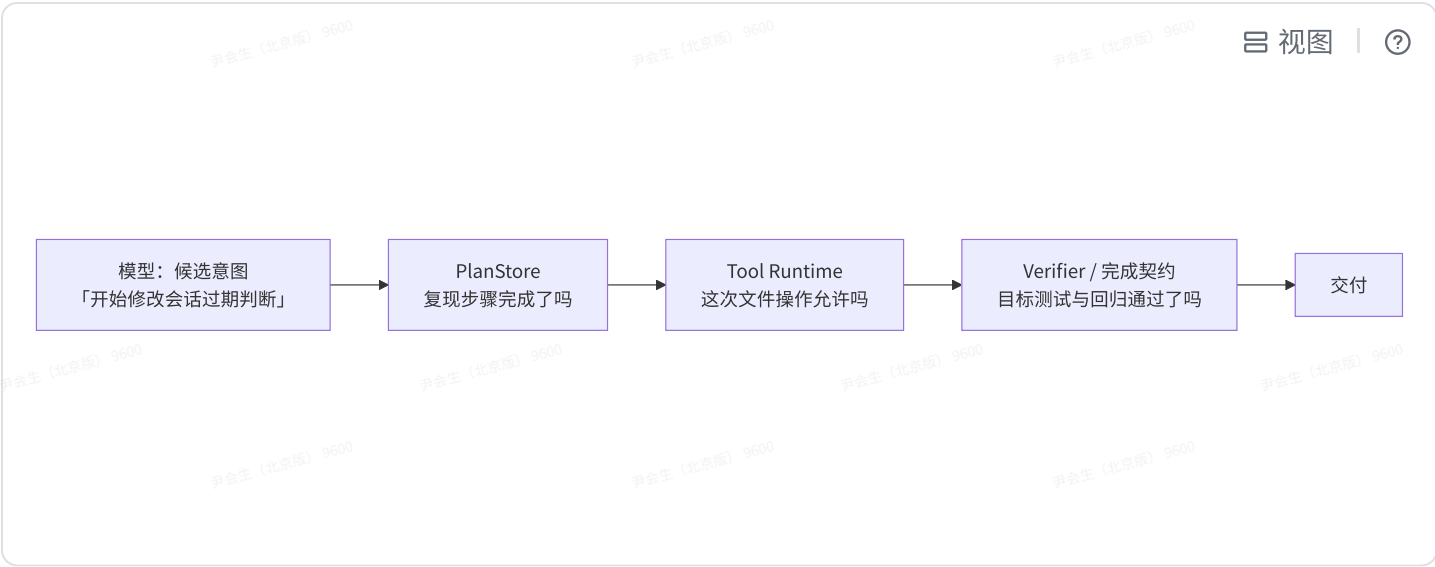


没有独立 Planner 的 Agent Loop，也能根据测试失败调整下一步——仅仅「下一步依赖工具结果」这一个条件，不足以说明必须引入计划层。

P09 · § 1.2 组件各管一段

对象	主要职责
Planner	提出子目标、依赖、下一步和修订建议
Plan / PlanStore	保存步骤、状态和修订历史，检查依赖与状态转换
pi Agent Loop	调用模型、处理 Tool Call、回写 Tool Result 并继续
Harness	装配模型、工具和计划组件，在执行接缝上落实约束
Tool Runtime	参数校验、权限检查、超时与工具执行
Verifier	检查实际证据是否满足当前验收条件

示意图：



一侧是模型提出的候选意图，一侧是程序给出的执行许可和验收结论。

P10 · § 1.3 什么时候不需要计划

- 成本：额外的模型调用、Token 消耗、延迟、状态分支
- 这些场景通常可以跳过显式计划：
 - 查询订单、读取单个文件这类单步工具调用
 - 固定字段抽取、格式转换和通知发送
 - 审批顺序由合规规则锁定，不允许模型增删节点
 - 延迟敏感，规划成本高于执行成本
 - 高风险动作，不允许模型动态改变步骤

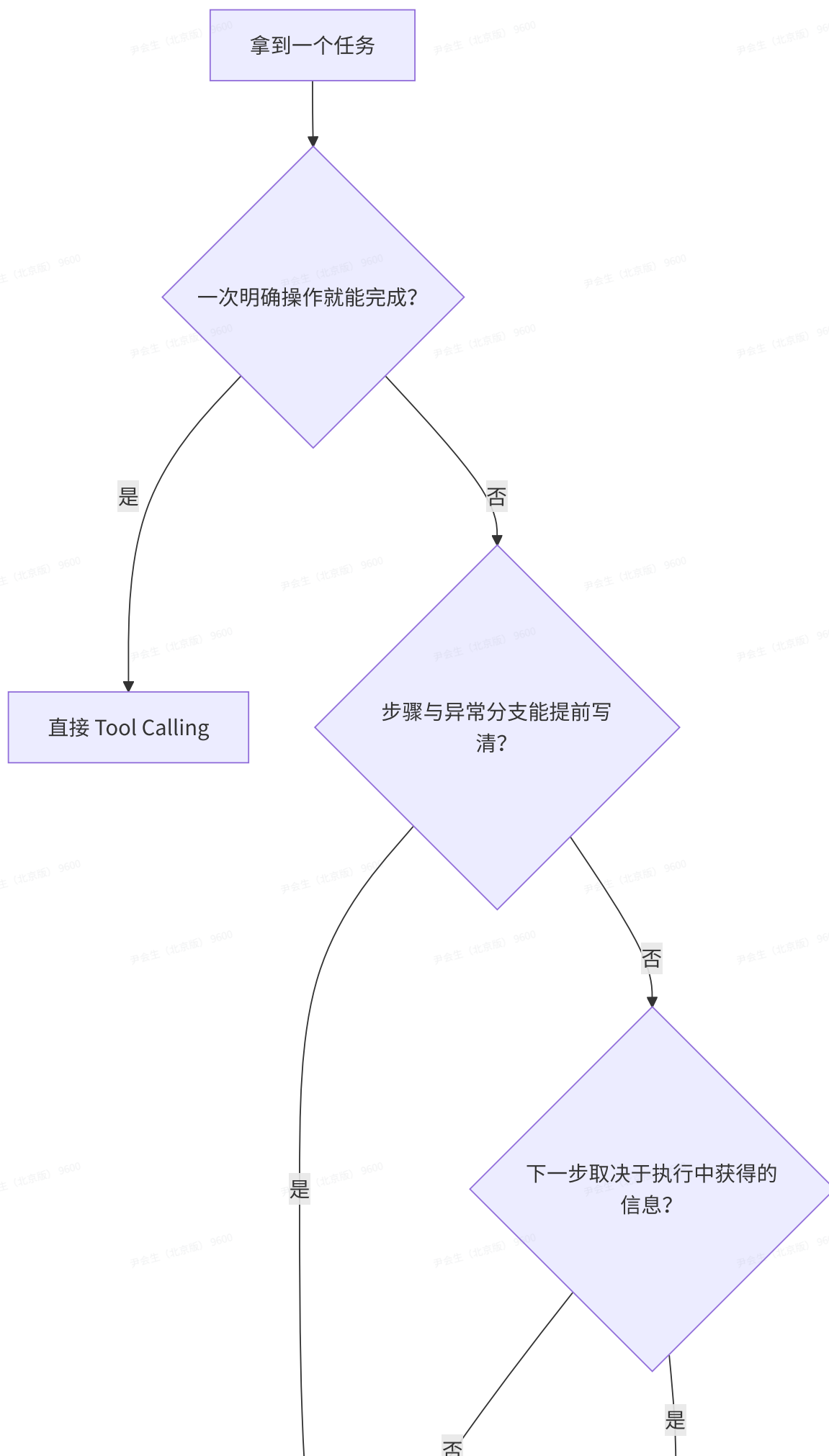
任务是否需要持续追踪多个子目标、依赖和验收条件。需要就拆，不需要就保持简单。

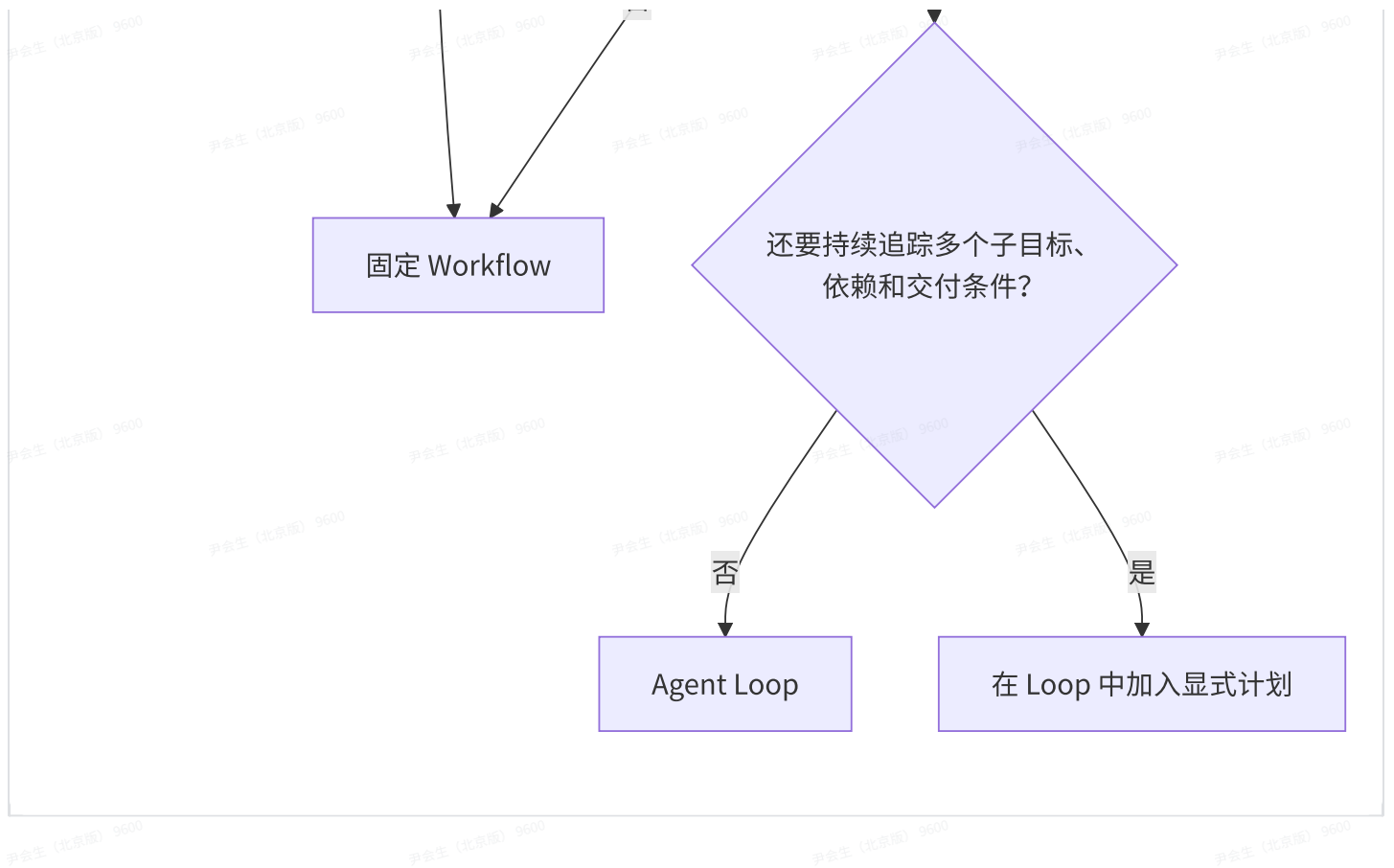
P11 · § 1.4 四种执行方式怎么选

任务特点	优先选择	例子
一次明确操作就能完成	直接 Tool Calling	查询某个订单状态
步骤与异常分支能提前写清	固定 Workflow	固定规则的数据导入

下一步取决于执行中获得的信息	Agent Loop	调查登录测试为什么失败
还要持续追踪多个子目标、依赖和交付条件	在 Loop 中加入显式计划	跨模块修复并完成分层验证

示意图：





易混点两条——固定 Workflow 也能含条件分支与重试；显式计划是任务管理能力，独立 Planner 只是组件拆分方式。

P12 · § 1.5 计划有两种形态

	一次性计划	动态计划
适用	目标明确、步骤稳定	路径无法在开始时确定、工具结果持续改变下一步
骨架	执行前形成完整骨架，过程中更新进度	初始版本保留未知，先安排诊断与形成假设
例子	3.1 的代码理解任务	本节这个修复任务
步骤由谁定	模型针对本次任务生成（规则固定则写 Workflow）	模型生成，随后按证据细化

- 「一次性」描述的是默认规划策略，不是禁止异常处理
- 本节定位：开始时只知道要「定位、修复、验证」，不知道根因在哪 → **需要动态计划**
- 本次真实运行：模型读完源码就直接锁定了根因（日期粒度截断），**一次走通，计划修订 0 次**

下一步是否取决于执行中获得的信息，以及是否需要持续追踪子目标、依赖和验收条件。

P13 · § 二 计划写成什么样 2.1 为什么「分析、修改、测试」不够

模糊动作	可验收子目标	可接受的证据
分析代码	确认失败入口、相关模块和一个可证伪的假设	文件位置、调用链、失败断言摘要
修改代码	完成符合边界的最小修改	受控 Diff、修改理由
运行测试	目标用例通过，且没有新增失败	命令、退出码、测试范围
修复缺陷	原问题消失，并存在防回归验证	修复前后对比、回归结果

放到本案例：

如果写成动作	改成可验收之后
「复现失败」	在 <code>scope=target</code> 上拿到退出码 1，并保留失败断言的期望值与实际值
「修一下」	产生当前代码版本上、允许修改范围内的最小 Diff
「跑测试」	<code>target</code> / <code>boundary</code> / <code>regression</code> 三个范围的退出码都为 0

左列问「做了什么」，右列问「完成后系统产生了什么新事实，以及用什么证明这个事实」。

P14 · § 2.2 一份计划至少装什么

代码块

```
1 type StepStatus =
2   | "pending" | "in_progress" | "completed"
3   | "blocked" | "skipped" | "failed";
4
5 interface Evidence {
6   id: string; stepId: string; toolCallId: string;
```

```
7 artifactVersion: string; // 产生证据时的代码版本
8 kind: "tool" | "test" | "diff" | "inspection";
9 summary: string; payload: Record<string, unknown>;
10 }
11
12 interface PlanStep {
13   id: string;
14   objective: string; // 完成后应产生的新事实
15   dependsOn: string[];
16   successCriteria: string[]; // 事前定义
17   status: StepStatus;
18   evidenceIds: string[]; // 事后收集
19   attempts: number; lastError?: string;
20 }
21
22 interface Plan {
23   version: number; goal: string; constraints: string[];
24   steps: PlanStep[]; revisionCount: number; maxRevisions: number;
25 }
```

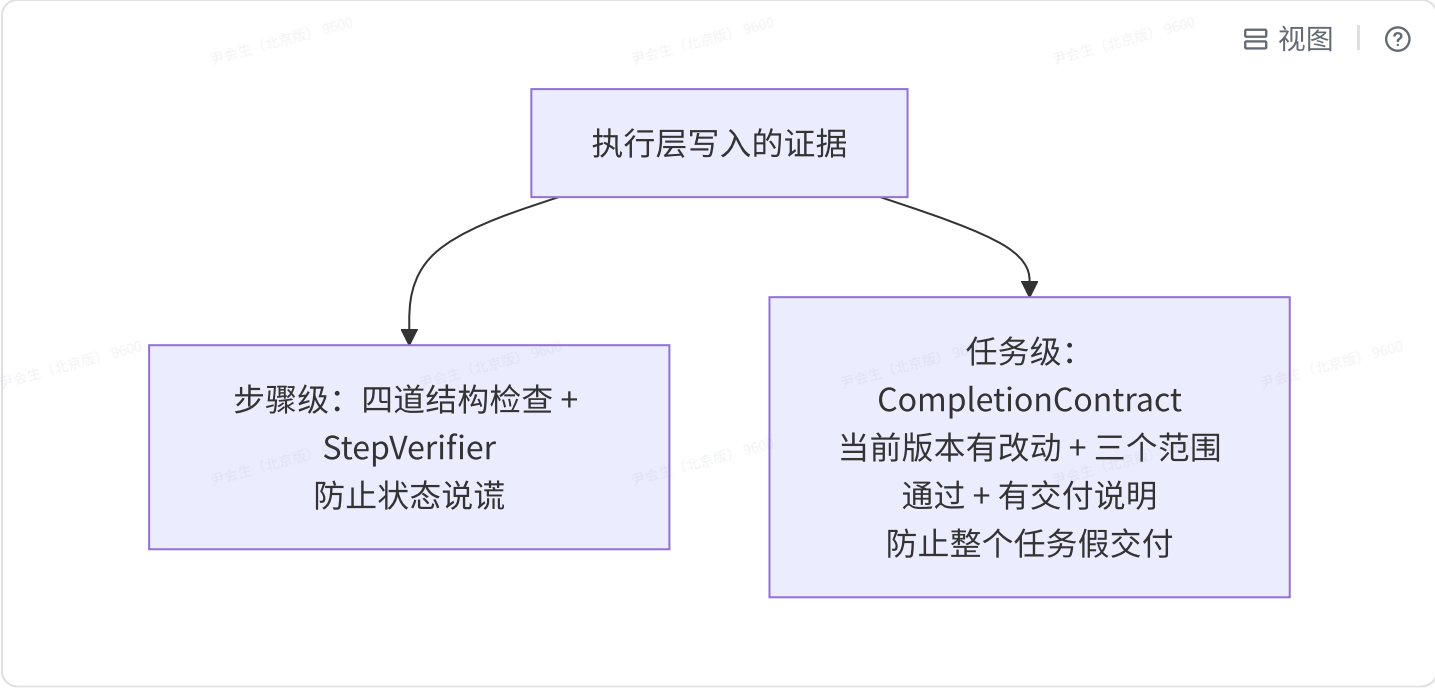
- `objective` 写结果，不写动作：「确认根因」而不是「分析一下」
- `successCriteria` 事前定义，`evidenceIds` 事后收集，**两者分开存放**
- `artifactVersion` 把证据钉在某个代码版本上，改过代码后旧证据自动失效
- `status` 有六个值，不是一个布尔「完成 / 未完成」
- `constraints` 和 `maxRevisions` 是计划自带的边界

这些类型是本节新增的应用代码，pi 没有自带这一层。

P15 · § 2.3 完成标准在前，证据在后

证据	能证明	不能证明
<code>scope=target</code> 退出码 1	复现步骤完成（原问题确实存在）	修复完成
<code>scope=target</code> 退出码 0，且版本为当前	修复验证完成	边界与回归也通过
三个范围退出码都为 0，且版本为当前	本次修复在约定范围内可交付	仓库其它区域没有回归

示意图：

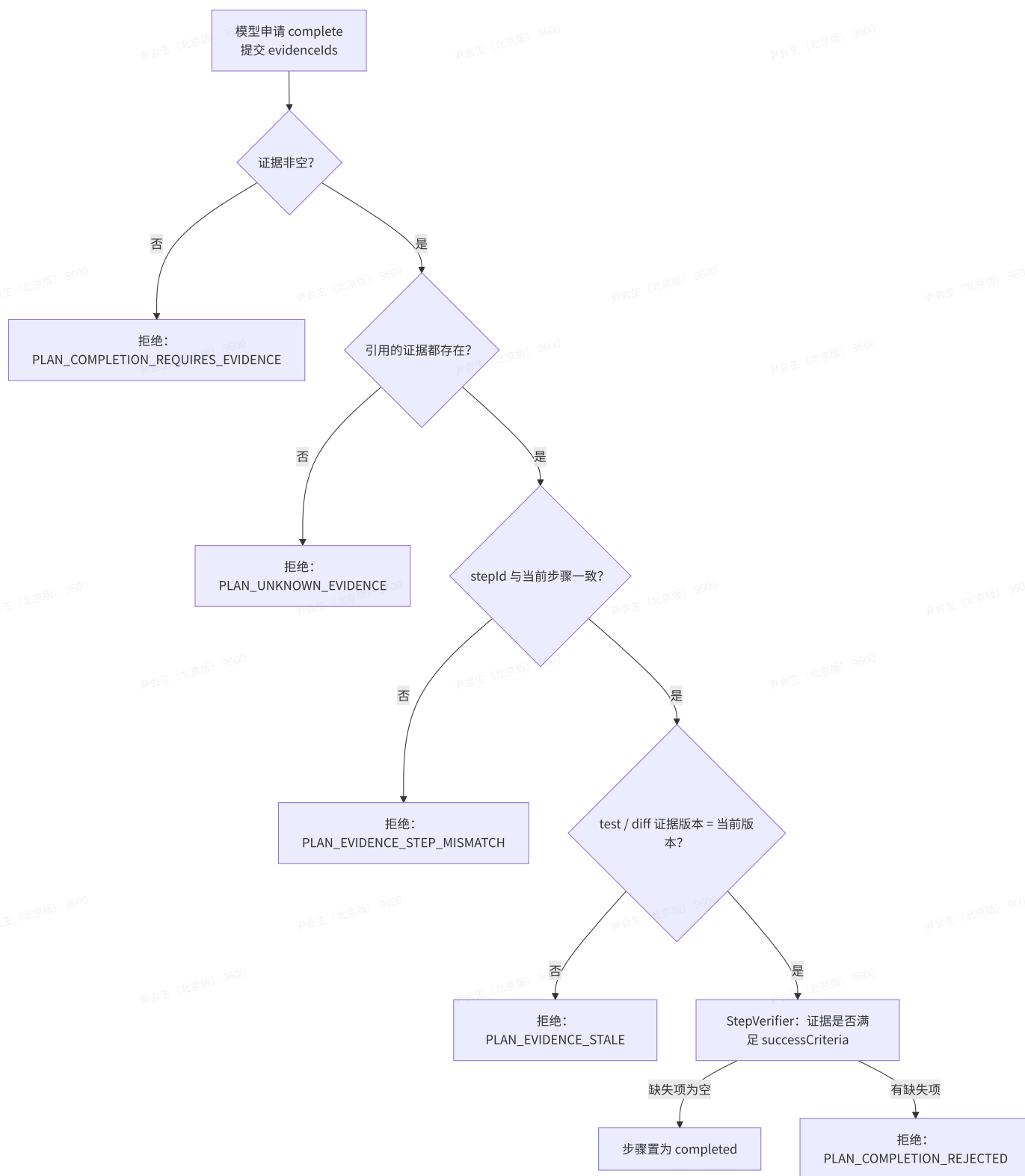


状态不能由退出码直接映射——是「复现完成」还是「修复被拒」，取决于这一步事前写下的标准。

P16 · § 2.3 证据只能引用，不能自述

- 证据由执行层写入，模型只能引用 id；Runtime 记下的退出码和代码版本才算
- 一次 `complete` 申请要依次通过四道检查，全部通过后才交给 `StepVerifier` 做语义检查

示意图：

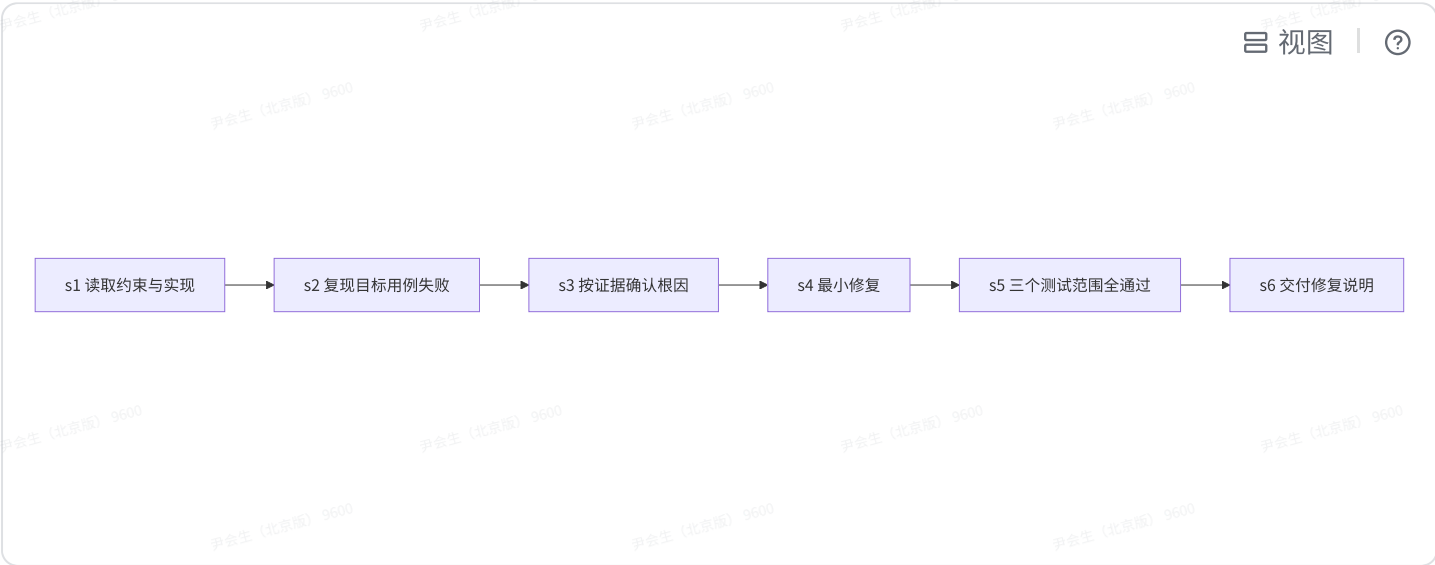


这条拒绝是保证了「每一步为什么算完成」可以被追溯。

P17 · § 2.4 案例的六步拆解

步骤	目标（完成后产生的新事实）	完成标准	证据条数
s1	确认登录入口、会话实现与允许修改范围	能指出过期判定在 <code>session-policy.ts</code> ，并说明调用它的位置	4
s2	复现目标用例失败	<code>scope=target</code> 退出码为 1，失败断言的期望与实际都有记录	1
s3	按证据确认根因	指出具体实现缺陷（ <code>slice(0,10)</code> 截断到日期），证据来自源码读取而非猜测	1
s4	完成允许范围内的最小修复	当前代码版本上产生受控 Diff	1
s5	三个测试范围全部通过	<code>target</code> / <code>boundary</code> / <code>regression</code> 各自退出码为 0	3
s6	交付修复说明	<code>artifacts/login-fix.md</code> 写入成功，且写明未验证项	1

示意图：



`s5` 是一条步骤、三个证据；六步里没有「修改密码校验」这一步——`s3` 是「确认根因」，不是「确认某个猜测」。

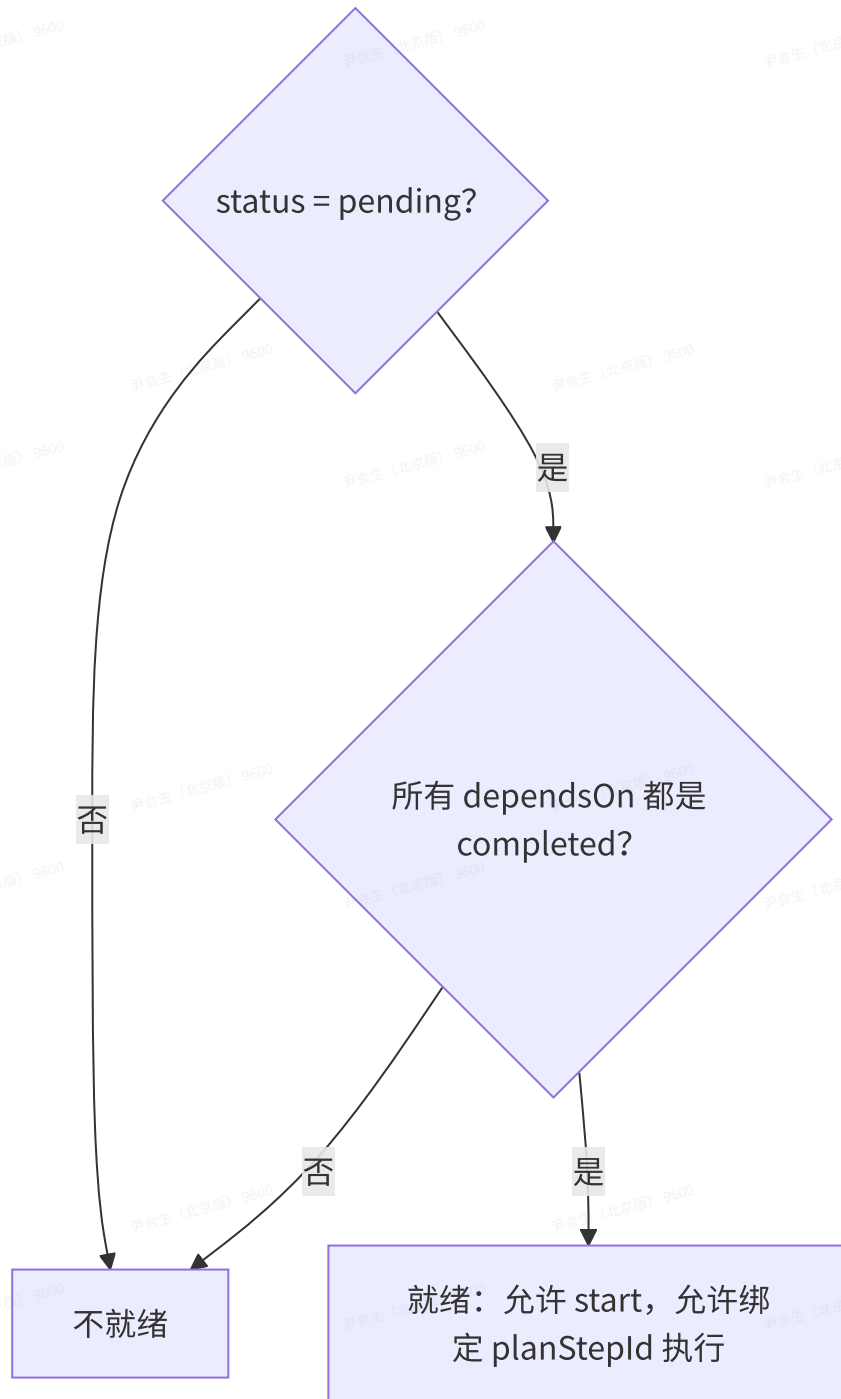
P18 · § 2.5 依赖决定谁先谁后

- 六类依赖：前置依赖 | 数据依赖 | 权限依赖 | 验证依赖 | 汇合依赖 | 互斥选择
- `ready` 由程序计算，不由模型声明：

```
1 readySteps(): PlanStep[] {  
2     return this.plan.steps.filter((step) =>  
3         step.status === "pending"  
4         && step.dependsOn.every((id) => this.requireStep(id).status ===  
5         "completed"),  
6     );  
}
```

- `skipped` $\neq$ `completed`：只要还有步骤依赖它，`skip()` 就直接拒绝，要求先做一次修订把依赖改接干净
- 依赖就绪不等于没有冲突：同时改同一文件、写同一张表仍需执行隔离

示意图：



模型的选择是候选意图，依赖检查才是执行许可。

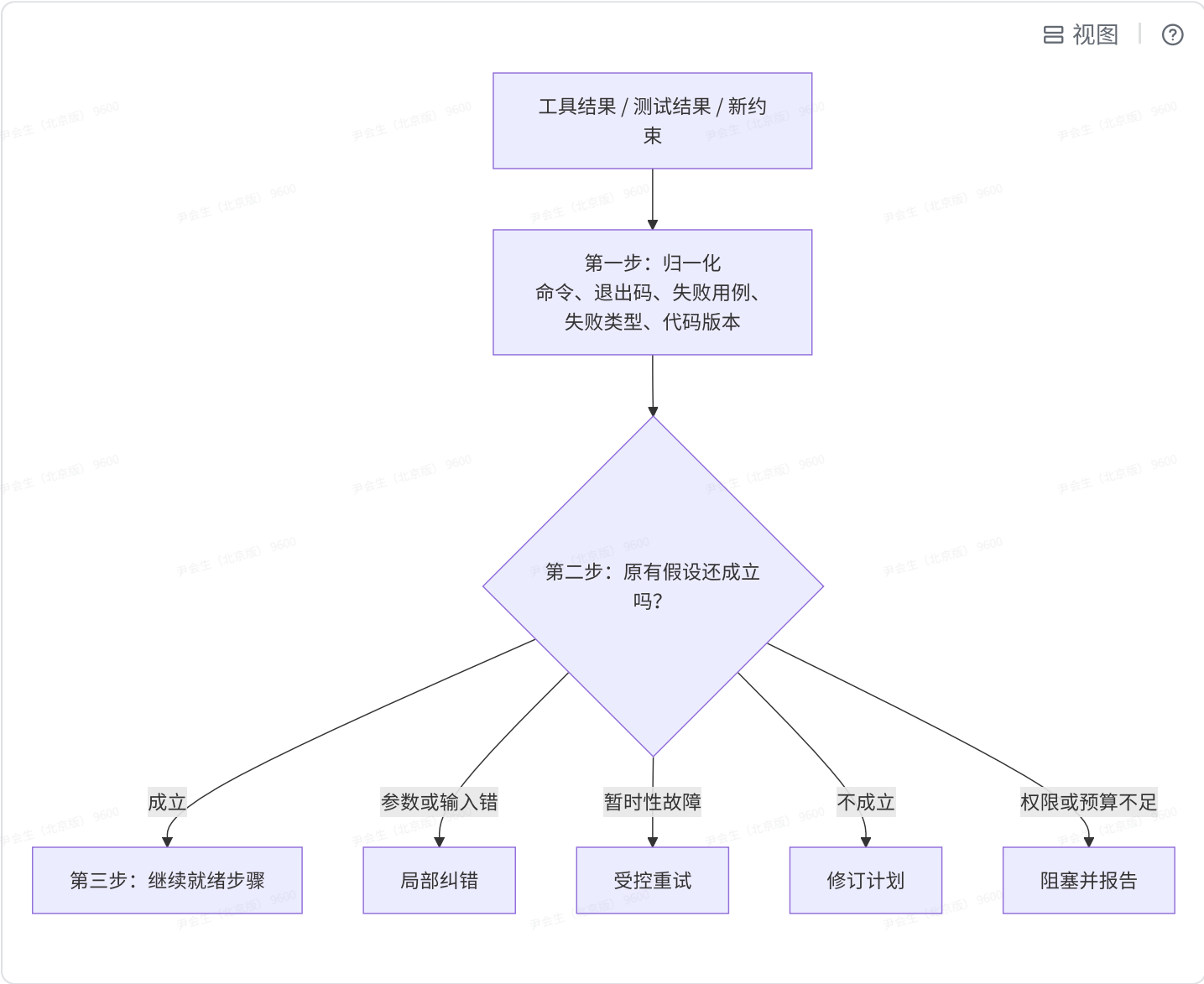
P19 · § 三 计划怎么变 3.1 收到偏差，先分流再动手

- 执行偏差是**观察到的事实**，「改计划」只是处置动作之一

新的观察	处置	是否修订计划

工具参数拼错	局部纠错：改正参数，保留原目标	不需要
目标文件不存在	先搜索实际位置；路径变了就局部修正，模块结构变了才修订	看情况
幂等查询短暂超时	受控重试，且不出既定预算	不需要
测试断言指向到期边界	先补证据，再判断原修复假设是否还成立	前提被推翻时
测试缺依赖，断言根本没执行	先恢复测试条件，必要时新增环境准备步骤	需要时新增步骤
用户中途补充约束	检查计划与已有 Diff，让冲突步骤和相关验收失效	需要
权限拒绝或预算耗尽	记录阻塞或停止原因	不能靠改计划绕过限制

示意图：



工具结果的第一问是——动作参数错了，还是计划前提错了。

P20 · § 3.1 测试失败先分类

排查的起点：

结果类型	下一步判断
assertion	测试到达业务断言但没满足预期，检查实现、测试预期或根因假设
environment	缺少依赖、配置或 fixture，断言根本没执行——先恢复测试条件，不能据此认定业务代码有错
timeout	检查卡住的位置与资源限制，区分死锁、慢测试和外部等待
infrastructure	执行器或 CI 基础设施失败，记录阻塞并按平台策略处理

- 本案例失败属于 `assertion`： `tests/session-policy.test.ts:10:10`，期望 `true` 实际 `false`
- 只有 `assertion` 这一类，才可能指向「根因在哪」这个假设
- 分类不能全信模型给的标签：`run_test` 只返回结构化事实，失败类型要靠读输出判断
-

有副作用且执行结果不明的操作，先确认是否已经生效，不能盲目重放。

P21 · § 3.2 什么才算一次修订

- 跟失败次数无关：
 - 把测试命令里的文件路径拼错了 → 局部纠错，改参数重跑，计划一个字都不用动
 - 失败断言真的推翻了「根因在哪」的判断 → 前提变了，必须修订
- 本次真实 Gateway 运行，计划修订 0 次；模型读完源码就直接锁定了根因，一次走通
- 它遇到过一次证据问题：想把登记在 `s1` 名下的证据用在 `s3` 上，被系统拒绝。那是证据归属纠正，不是修订
- 没有新信息时原样重试，不算有效修订

动态重规划这块能力由确定性测试覆盖，不把本次真实运行说成已经触发过动态重规划。

P22 · § 3.3 一次修订的完整记录

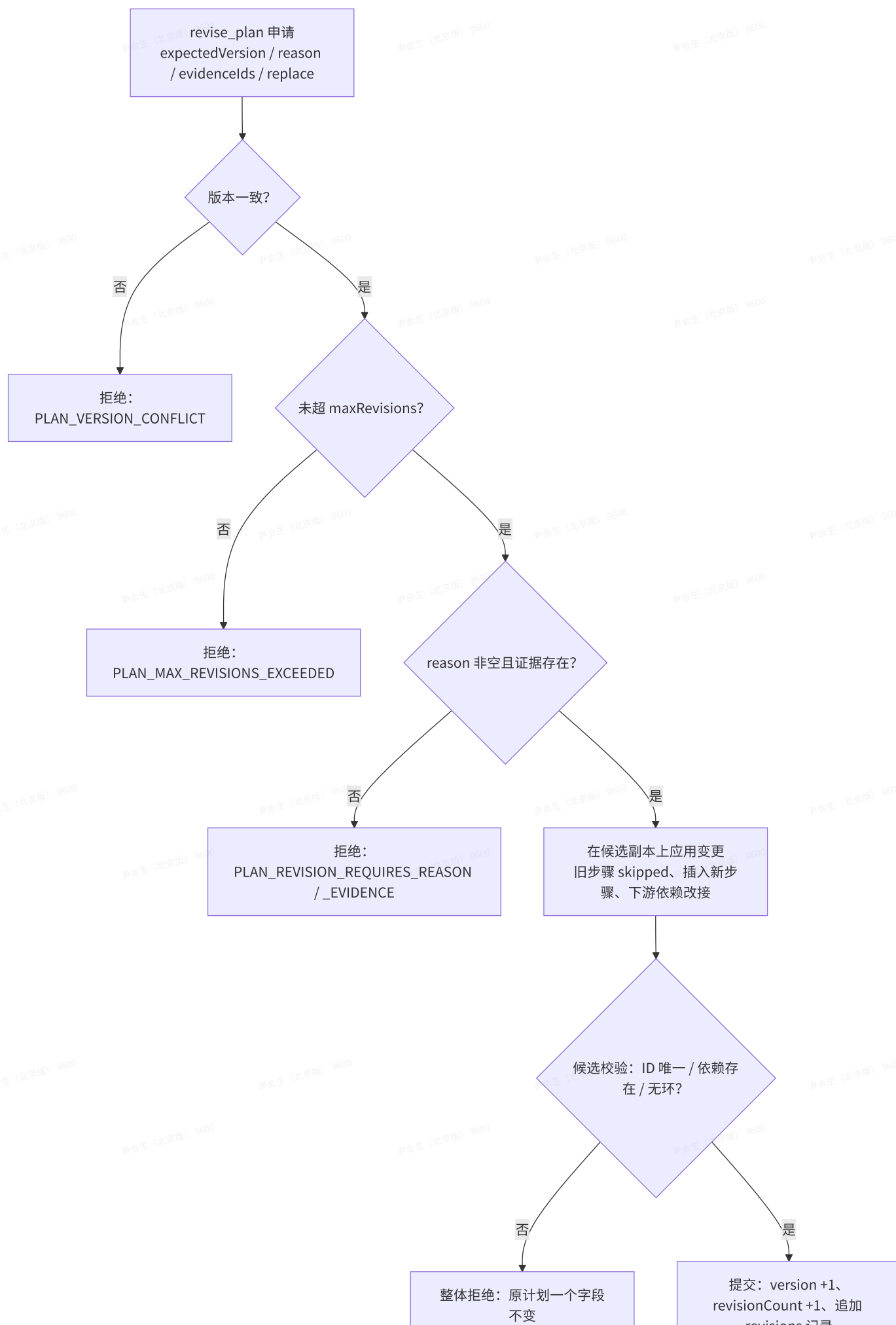
- 模型的申请（`revise_plan`）：

```
1 expectedVersion: 1
2 reason: 失败指向会话有效期判断，而不是密码校验
3 evidenceIds: [ev-3]
4 replace: modifyPasswordLogic → fixExpiryCheck
5   objective: 修正会话有效期判断
6   dependsOn: [reproduce]
7   successCriteria: [按到期时刻比较]
```

- 提交后快照：旧步骤 `skipped`（`replaced by fixExpiryCheck`）、新步骤插入、下游 `dependsOn` 自动改接、`revisions[0].changes` 记下 `replace:` 与 `rewire:`

规则	做法	不这么做会怎样
版本对齐	请求携带 <code>expectedVersion</code> ，与当前版本不一致就拒绝	基于旧版本的申请覆盖新计划
原子提交	替换、追加、依赖改接在候选副本上一次完成，校验通过才提交	中间状态里旧步骤已跳过、新步骤还没接上，验证步骤误判为就绪
预算上限	提交前检查 <code>maxRevisions</code>	失败循环里无限修订，永远不收敛

示意图：



关键是「候选副本」四个字——有环、重复 ID、依赖缺失都会让整次修订被拒绝。

P23 · § 3.4 修订只能改路径，不能改标准

- 模型可以调整路径，不能把「回归测试通过」改成「尝试运行过测试」，也不能删掉断言让任务看起来完成
- 用户中途补充约束（课堂示例：用户说「不要修改公共 API」）：
 - 更新约束版本 → 检查已有 Diff → 暂停尚未执行的接口改动 → 已违反新要求的改动进入修正与重新验证
 - 旧测试结果不能自动证明新约束已满足
- 有不可逆的外部副作用：记录事实并请求人工处理，**不能用改计划的方式假装撤销了操作**

计划可以变，完成标准不能变。

P24 · § 四 怎么落地代码 4.1 改动范围

文件	相对对照组的变化
model.ts	沿用 Gateway 模型配置，不动
run-context.ts	增加目标源码、目标用例、边界用例、回归范围
runtime.ts	工具沿用，新增代码版本计数与证据草稿产出
plan-store.ts（新增）	保存计划、检查依赖、校验证据、提交修订
plan-tools.ts（新增）	创建、读取、更新、修订计划的契约与 Handler
completion-contract.ts（新增）	把对照组硬编码的完成判断提取为可注入契约
planning-prompt.ts（新增）	运行提示词与每轮替换的计划快照

pi-tools.ts	增加计划工具注册与 planStepId 的 Schema 扩展
loop-guard.ts	保留轮数与重复保护，增加计划门禁，完成契约改为注入
agent-runner.ts	装配计划工具、计划状态和运行提示词
main.ts	启动计划化 Loop，打印计划与证据摘要

pi 的 Agent Loop 一行没改；判定「有没有引入第二套 Loop」的标准很简单——模型调用、工具执行、结果回写这三个入口还在不在 pi 手里。

P25 · § 4.2 第一步：计划数据与计划工具

代码块

1

在现有工程 (model.ts、run-context.ts、runtime.ts、pi-tools.ts、loop-guard.ts、agent-runner.ts) 上增加最小计划 planning 能力。

2

先读这些文件和实际 pi 类型，列出扩展位置，不要引入第二套 Agent Loop。

3

4

4

新增 plan-store.ts:

5

1. 定义 StepStatus、PlanStep、Plan、Evidence、PlanRevision;

6

2. PlanStore 提供 readySteps / start / complete / fail / block / skip / revise;

7

3. 只有依赖全部 completed 才允许开始; skipped、failed 都不满足依赖;

8

4. complete 必须引用已存在的证据，证据要属于该步骤，并通过 StepVerifier;

9

5. 测试与改动证据必须产自当前代码版本，旧版本的通过结果不能用于交付;

10

6. revise 携带 expectedVersion、reason、evidenceIds，整体校验通过才提交;

11

替换步骤时保留旧步骤并标记 skipped，下游依赖原子改接到新步骤;

12

7. 提交前检查修订次数上限; 步骤 ID 唯一、依赖存在、依赖图无环;

13

8. 先实现单进程内存版本。

14

15

新增 plan-tools.ts: create_plan / get_plan / update_plan_step / revise_plan 的契约与 Handler。

16

模型只能提出操作，状态只能由 Handler 修改; complete 只是完成申请。

17

18

交付类型、Handler、工具注册，以及覆盖上述规则的测试。

19

不要修改模型配置，不要新增第二个工具执行入口。

类怎么组织、重复代码怎么提取交给 AI；你要盯住的是这张规则清单有没有被真正落实。

P26 · § 4.3 接进 pi 的四个 Hook

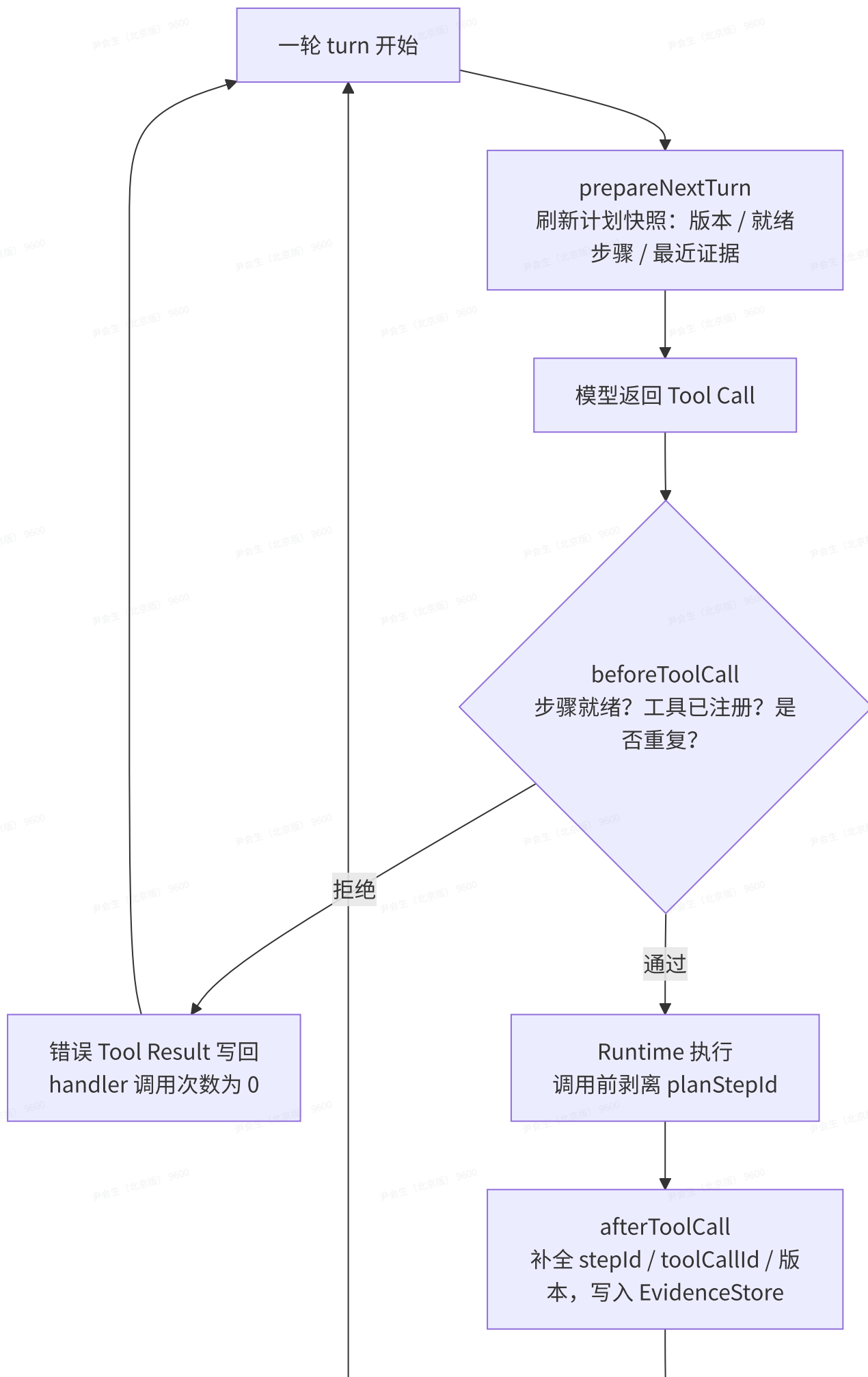
执行位置	本节增加的行为
<code>beforeToolCall</code>	校验动作绑定的步骤是否就绪；继续执行原有重复保护
<code>afterToolCall</code>	把 Runtime 的真实结果关联到步骤，形成可引用证据
<code>prepareNextTurn</code>	刷新计划版本、就绪步骤和最近证据摘要
<code>shouldStopAfterTurn</code>	保留原停止规则，改用注入的完成契约检查交付

- 第一轮就注入规划规则与「尚未创建计划」的状态，否则模型不知道该先调 `create_plan`
- 每轮替换快照，不把历次完整计划反复追加进上下文
- 三件必须守住的边界：
 - 原始 Tool Result 仍由 `pi` 正常回写，快照不替代工具结果
 - 仓库文本和工具输出按待分析数据处理，不借提示词位置变成指令
 - `planStepId` 在调用 Runtime 之前剥离，它是计划校验字段，不是业务参数

代码块

```
1  把已完成的 PlanStore 接入现有 pi runAgentLoop。
2
3  保留流式输出、取消信号、最大轮数、重复调用保护和原工具权限。
4  1. 给工具 Schema 增加可选 planStepId；写文件、改代码、跑测试必须携带，
5    由 beforeToolCall 校验，适配层在调用 Runtime 之前剥离该字段。
6  2. Runtime 只产出证据草稿；afterToolCall 补全步骤 ID、toolCallId 和当前代码版本后写
   入 EvidenceStore。
7  3. prepareNextTurn 每轮刷新计划快照，不重复堆积历史；第一轮就注入规划规则与初始状态。
8  4. 把 LoopGuard 中硬编码的 login-flow.md 完成判断提取为 CompletionContract：
9    保留原代码理解任务的契约；
10   为修复任务新增「当前版本有改动 + 目标用例、边界用例、约定回归全部通过 + 有交付说明」。
11  5. 模型不调用工具不代表完成；证据不足沿用 Follow-up 机制继续。
12  6. 预算耗尽、取消、阻塞必须输出真实停止原因，不能标为 completed。
13
14  依据仓库实际 Hook 类型实现，交付最小 Diff 和验证结果。
```

示意图：



代码块

```
1  你是 Codebase Agent。根据任务规模决定是否使用显式计划：
2  简单查询直接调用工具；复杂修复先创建或读取计划。
3
4  计划步骤描述可验收结果，列出 dependsOn 和完成标准。
5  未知根因先安排诊断，不把猜测写成已经确认的修改目标。
6  执行受计划管理的动作时携带对应 planStepId。
7
8  每次工具返回后：
9    - 判断结果支持还是推翻当前假设；
10   - 条件未变化就继续就绪步骤，局部参数错误先局部修正；
11   - 假设或约束变化时，引用证据申请 revise_plan；
12   - 遇到权限或预算限制，报告阻塞，不绕过规则。
13
14  申请完成步骤时引用真实 evidenceIds。
15  不得降低验收标准，不得删除必要验证以提前结束。
16  最终报告包含改动、实际验证命令、结果和未验证项。
17  只输出必要判断与行动依据，不要求展开完整思维链。
```

具体是哪个文件、允许改哪个源码，都不在提示词里，由运行上下文限定——范围约束不依赖模型自觉。

P28 · § 4.5 三个容易写错的地方

1. 完成判断只看 `evidence.length > 0`
失败日志也是证据，只看数量会让修复步骤直接通过
2. 把 `skipped` 当成 `completed`
跳过验证之后下游照样就绪，任务在没验证的情况下交付
3. 只在 Prompt 里写「遵守依赖」
执行器看不到依赖，模型照样能不按步骤调用工具，依赖检查形同虚设
- 这三处就是下一轮验收的重点。

P29 · § 4.6 验收一：计划层规则（13 个用例）

测试	必须观察到的结果
依赖未完成时申请开始	PLAN_STEP_NOT_READY；readySteps() 只给出 inspect
必要前置被标记 skipped	PLAN_SKIP_REQUIRES_REWIRE；下游仍未就绪
没有证据申请完成	PLAN_COMPLETION_REQUIRES_EVIDENCE
引用不存在的证据	PLAN_UNKNOWN_EVIDENCE
证据不属于被完成的步骤	PLAN_EVIDENCE_STEP_MISMATCH
证据存在但不满足标准	PLAN_COMPLETION_REJECTED，例如修复验证拿到 exitCode=1，步骤留在 in_progress
引用修改前版本的通过结果	PLAN_EVIDENCE_STALE，不能用于交付
替换修改步骤	旧步骤保留为 skipped，verify.dependsOn 改接新步骤，changes 记下 replace: 与 rewire:
修订产生环	PLAN_DEPENDENCY_CYCLE，整次修订被拒，快照与修订前完全相等
修订缺理由或缺证据	PLAN_REVISION_REQUIRES_REASON / PLAN_REVISION_REQUIRES_EVIDENCE
以旧版本发起修订	PLAN_VERSION_CONFLICT，版本不变
超过修订预算	PLAN_MAX_REVISIONS_EXCEEDED，版本与步骤状态不变
创建计划时依赖不存在	PLAN_UNKNOWN_DEPENDENCY

允许「用预期失败完成复现步骤」——一份失败结果能不能证明完成，取决于这一步的 successCriteria 写的是什么，跟退出码的符号无关。

P30 · § 4.7 验收二：Loop 集成（5 个用例）

测试	必须观察到的结果
完整闭环：从初始假设到证据修订	计划版本 v1 → v2，modifyPasswordLogic 变 skipped，verify_regression 依赖改接为目标与边界两项；源码真的被改写；交付说明真的落盘

受约束的动作缺少 <code>planStepId</code>	<code>PLAN_STEP_REQUIRED</code> , <code>apply_patch</code> 的 handler 调用次数为 0
绑定未就绪步骤的调用	<code>PLAN_STEP_NOT_READY</code> , <code>run_test</code> 的 handler 调用次数为 0
引用不存在的证据申请完成	<code>PLAN_UNKNOWN_EVIDENCE</code> , 步骤留在 <code>in_progress</code>
证据不足时模型给出 Final	注入 <code>FINAL_REJECTED</code> Follow-up 继续；本轮上限用尽后停止原因是 <code>MAX_TURNS_EXCEEDED</code>

- 前两条要连起来看：**校验发生在执行之前，被拦住的动作 handler 根本没被调用**——这是「依赖检查是真门禁，不是提示词」的唯一硬证据
- 边界：重复保护、最大轮数、取消信号沿用 3.1 的 LoopGuard，本节没有新增用例
- `npm test` 共 18 个用例：13 覆盖计划层规则，5 覆盖 Loop 集成；项目只有 `npm test` / `npm run build` / `npm start`

脚本测试只证明规则成立，接线是否真的通了要看真实运行。

P31 · § 4.8 真实运行怎么留证据

代码块

- 1 验证本次计划能力改动：
- 2 先运行计划规则的确定性测试：依赖未就绪、证据缺失、证据不属于该步骤、
- 3 旧版本证据、有环的修订、旧版本修订、超出修订预算。
- 4 再沿用现有 Gateway 真实运行一次登录修复任务，记录实际命令、退出码、计划修订与交付结果。
- 5 检查受约束的调用缺少 `planStepId` 时，handler 是否真的没有执行。
- 6
- 7 不得为通过验收而删除断言、降低标准或伪造工具结果。
- 8 区分已实现、已验证和未验证项；环境阻塞时给出具体原因。
- 9 最后说明相较对照组修改了哪些文件，以及这些改动各解决什么问题。

P32 · § 4.9 学到什么程度算掌握

站	要能回答
一、要不要计划	这个任务该用 Tool Calling、固定 Workflow、Agent Loop，还是加显式计划
二、计划写成什么样	子目标怎么写成可验收状态，证据由谁提供，哪些步骤当前就绪

三、计划怎么变	哪种失败只需局部纠错，哪种变化必须走修订，修订为什么不能降标准
四、怎么落地与验收	怎么在 <code>pi</code> 的接缝上验证拦截、证据关联和完成条件

给 AI 一份明确的规格，让它补实现，再用这些反例检查结果——代码由谁写可以变，执行边界和验收结果要由你负责。

P33 · 本节小结

- Planning 给 Harness 提供**可执行、可验证、可修订**的任务状态
- 计划表达的是当前的行动假设；成熟的 Agent 不追求第一次计划完全正确，它追求的是：遵守当前有效的约束，历史可追踪，证据可验证，用最小修订调整下一步
- 计划、步骤、工具结果和修订记录目前都放在内存里，进程中断或上下文压缩之后会丢

下一节进入 State Machine 与 Checkpoint，把这些执行状态持久化，并解决任务中断、恢复、复现和人工接管。